CBSE Polymorphism

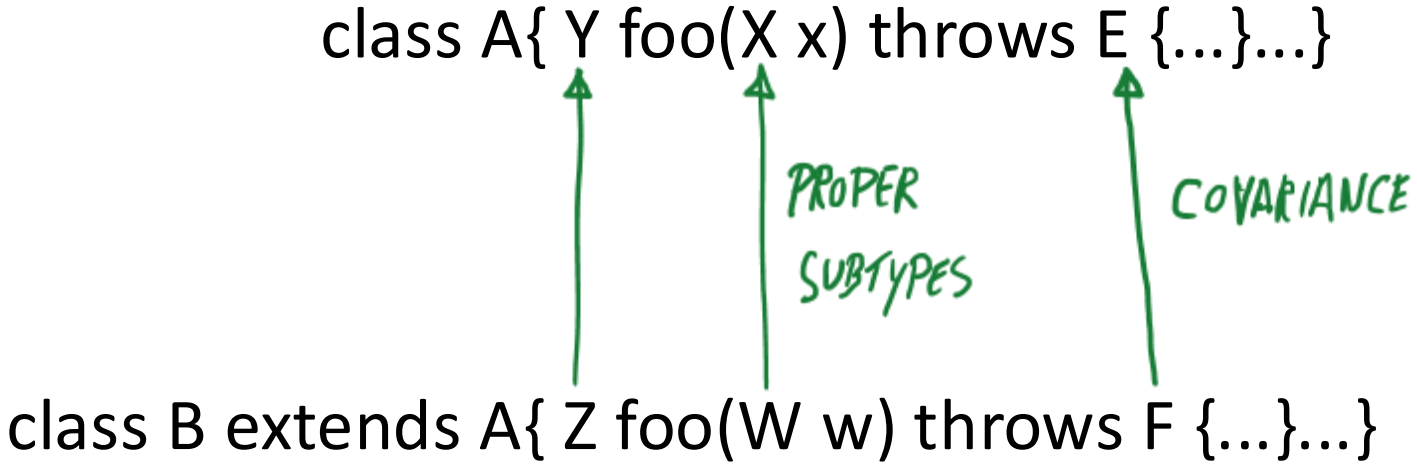
Contracts

A contract is represented by an interface with pre- and post-conditions attached to every operation in the library.

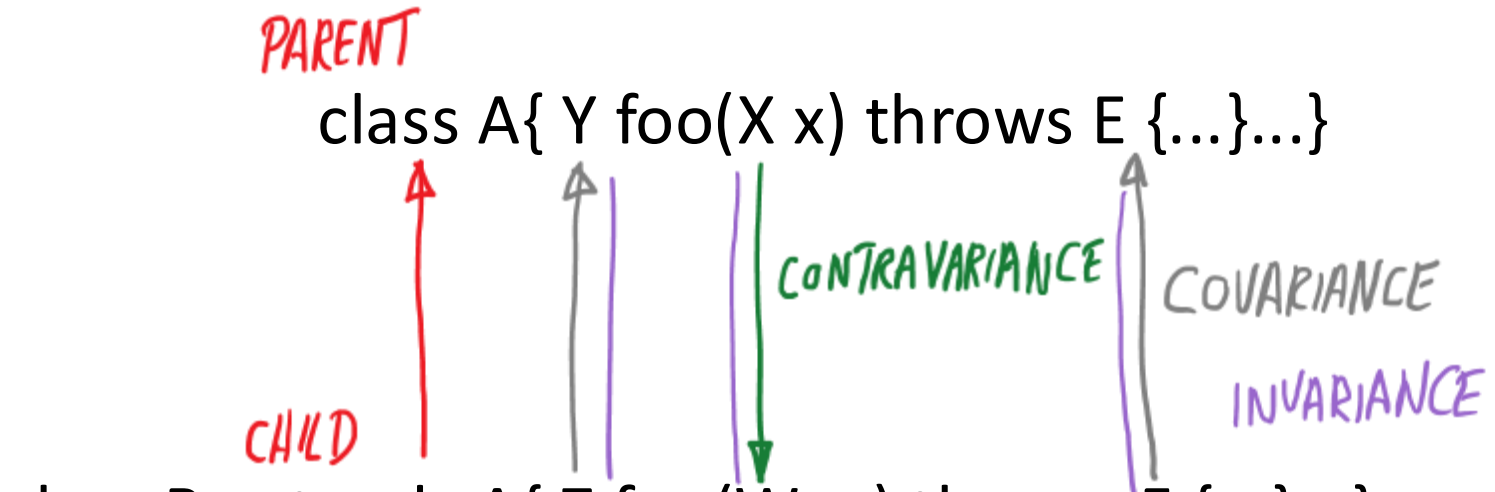
Class conformance [Liskov]:

- A module Y is conformant to module X if it can safely replace X in the system
- A subclass of Y is conformant to a superclass X if all objects of Y can safely replace all objects of X

Types in Contracts



OOP Syntactic Substitutability Rules



PROVIDER OF SUBCLASS B MAY EXPECT LESS THAN A GUARANTEES

Example

```
public interface View {
    Model getModel();

    void setModel(Model model);
}
```

COVARIANT

INVARIANT

```
public interface TextView extends View {
    @Override
    TextModel getModel();

    @Override
    public void setModel(Model model);

}
```

Types in Contracts

- Video Example.

Contract beyond Types

Hoare triplets:

{PRE CONDITION} OPERATION {POST CONDITION}

"Demand no more, provide no less":

CONTRACT PRE-CONDITION MUST IMPLY PROVIDERS PRECONDITION

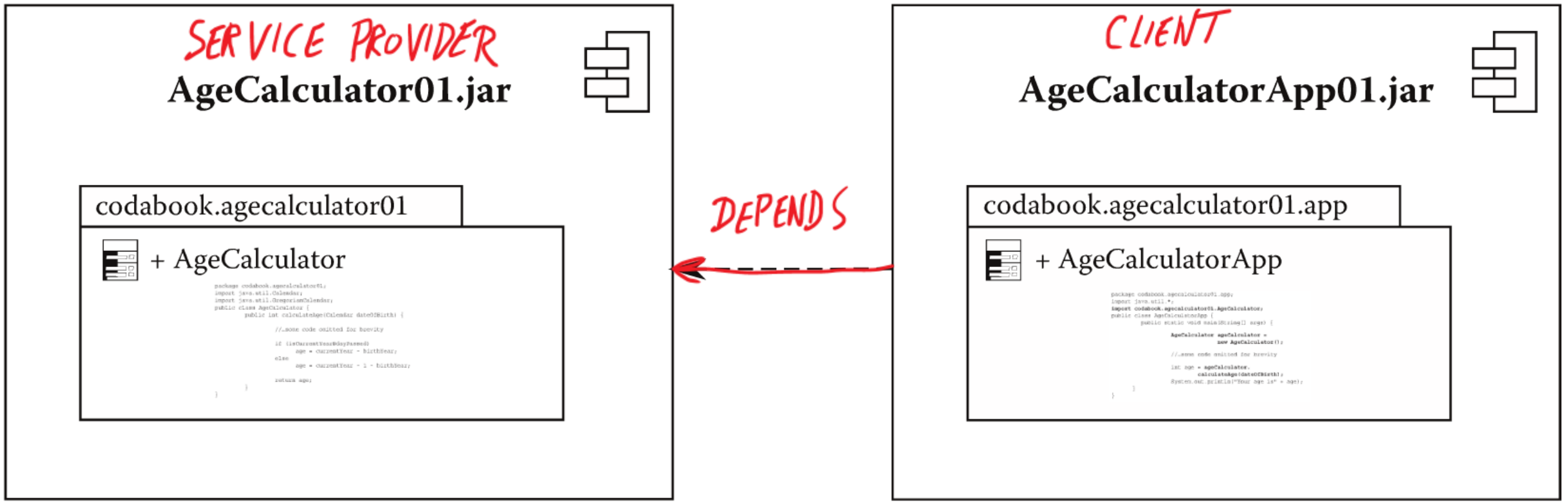
PROVIDERS POSTCONDITION MUST IMPLY CONTRACT POSTCONDITION

A CLIENT MAY EXPECT MORE THAN IS REQ. BY CONTRACT PRECONDITION AND EXPECT LESS THAN REQ. BY POSTCONDITION

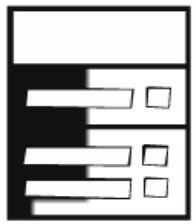
THAT IS, A PROVIDER CAN ASSUME WEAKER PRECONDITIONS AND STRONGER POSTCONDITIONS THAN CONTRACT.

Java and Components

JARs and Packages



codabook.agecalculator01.app



+ AgeCalculatorApp

```
package codabook.agecalculator01.app;
import java.util.*;
import codabook.agecalculator01.AgeCalculator;
public class AgeCalculatorApp {
        public static void main(String[] args) {

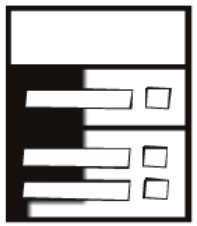
                AgeCalculator ageCalculator =
                        new AgeCalculator();

                //…some code omitted for brevity

                int age = ageCalculator.
                        calculateAge(dateOfBirth);
                System.out.println("Your age is" + age);
        }
}
```

BUILD TIME DEPENDENCY !!!

codabook.agecalculator01



+ AgeCalculator

```
package codabook.agecalculator01;
import java.util.Calendar;
import java.util.GregorianCalendar;
public class AgeCalculator {
        public int calculateAge(Calendar dateOfBirth) {

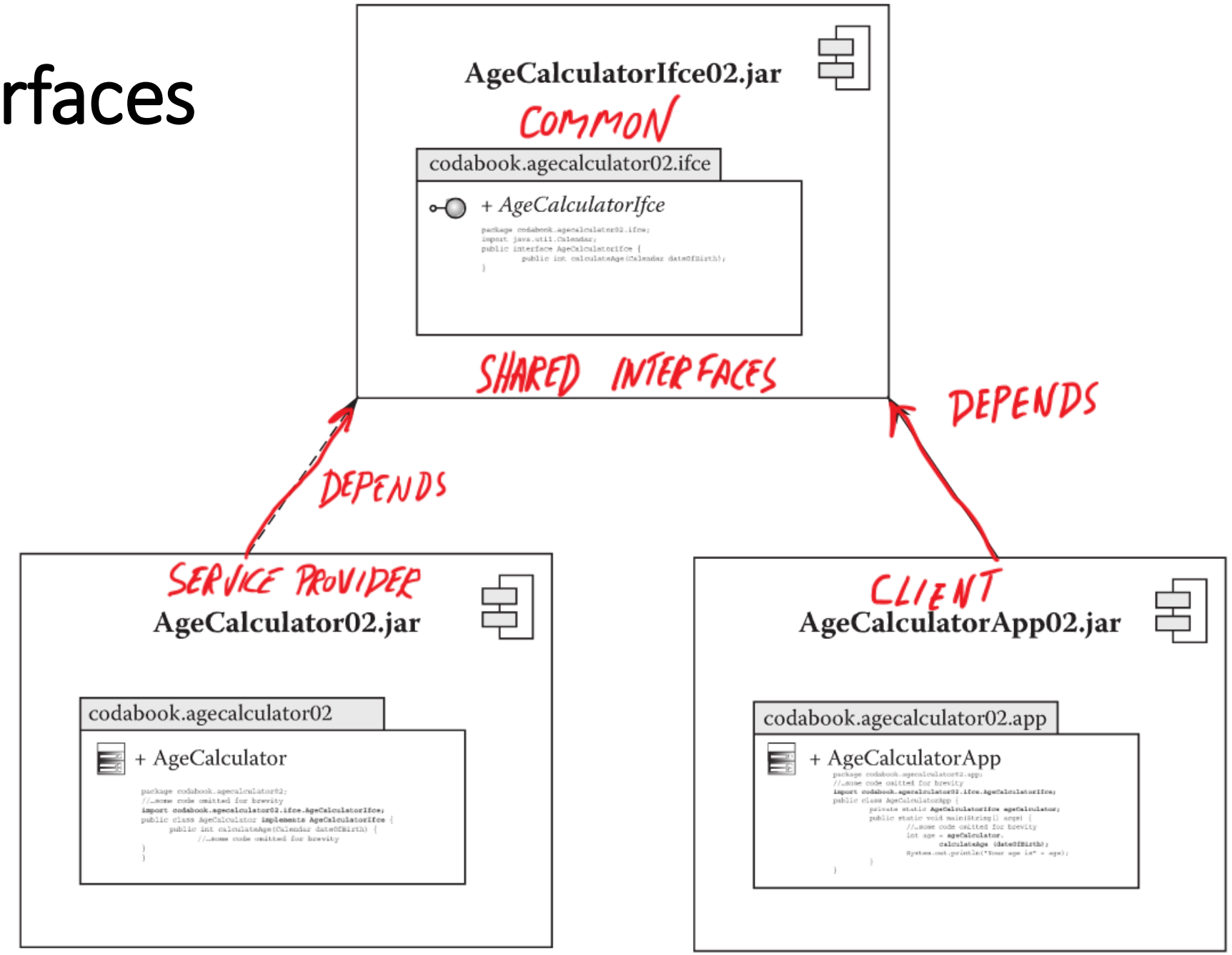
                //…some code omitted for brevity

                if (isCurrentYearBdayPassed)
                      age = currentYear - birthYear;
                else
                      age = currentYear - 1 - birthYear;

                return age;
        }
}
```

SIMPLE SERVICE IMPL.

Interfaces


AgeCalculatorIfce02.jar
COMMON
codabook.agecalculator02.ifce
+ AgeCalculatorIfce
SHARED INTERFACES
DEPENDS
DEPENDS
SERVICE PROVIDER
AgeCalculator02.jar
codabook.agecalculator02
+ AgeCalculator
CLIENT
AgeCalculatorApp02.jar
codabook.agecalculator02.app
+ AgeCalculatorApp

codabook.agecalculator02.ifce



+ *AgeCalculatorIfce*

```
package codabook.agecalculator02.ifce;
import java.util.Calendar;
public interface AgeCalculatorIfce {
        public int calculateAge(Calendar dateOfBirth);
}
```

codabook.agecalculator02.app



+ AgeCalculatorApp

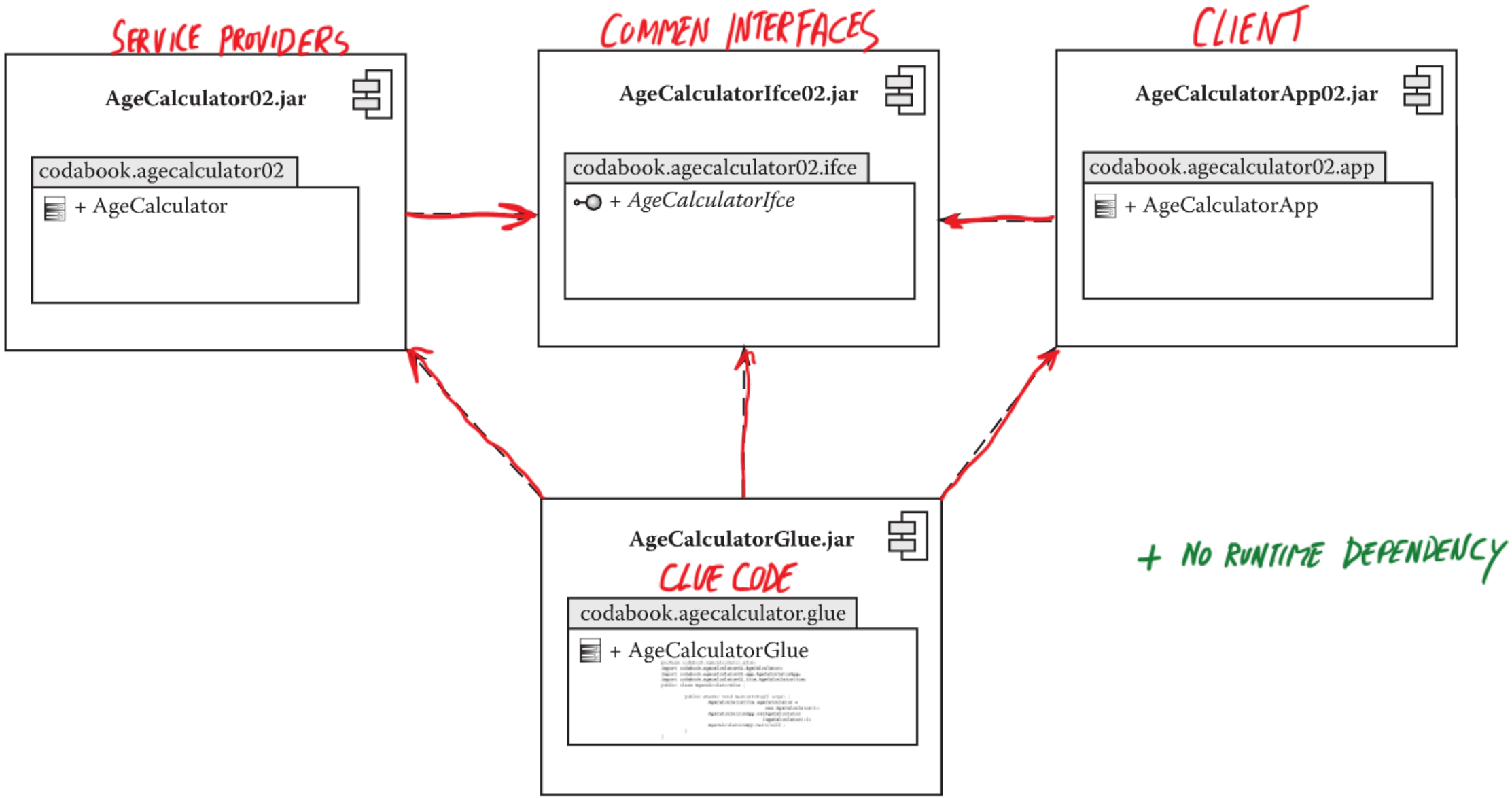
```
package codabook.agecalculator02.app;
//…some code omitted for brevity
import codabook.agecalculator02.ifce.AgeCalculatorIfce;
public class AgeCalculatorApp {
        private static AgeCalculatorIfce ageCalculator;
        public static void main(String[] args) {
                //…some code omitted for brevity
                int age = ageCalculator.
                        calculateAge (dateOfBirth);
                System.out.println("Your age is" + age);
        }
}
```

NO BUILD TIME DEPENDENCY

NULLPOINTER EXCEPTION

RUNTIME DEPENDENCY !!!

Assembly Clue



codabook.agecalculator.glue



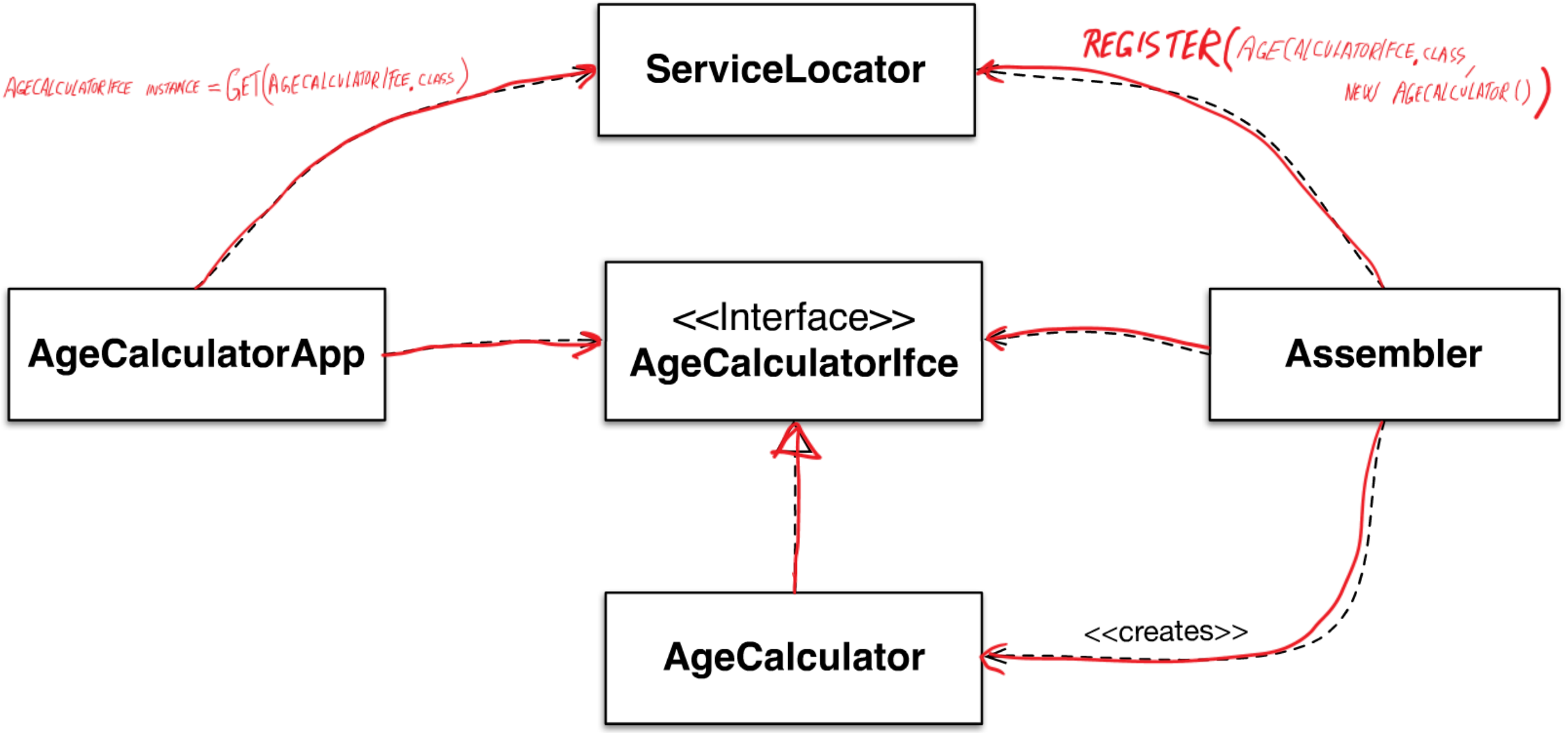
+ AgeCalculatorGlue

```
package codabook.agecalculator.glue;
import codabook.agecalculator02.AgeCalculator;
import codabook.agecalculator02.app.AgeCalculatorApp;
import codabook.agecalculator02.ifce.AgeCalculatorIfce;
public class AgeCalculatorGlue {

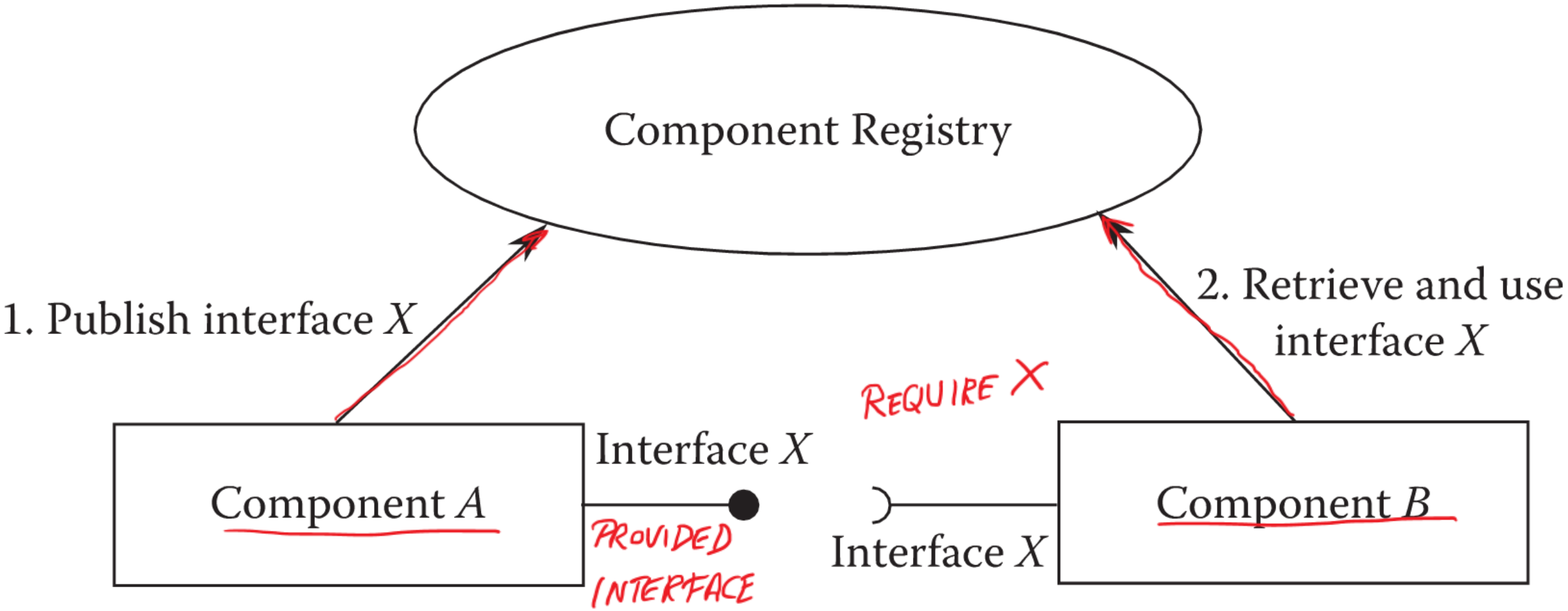
        public static void main(String[] args) {
                AgeCalculatorIfce ageCalculator =
                                new AgeCalculator();
                AgeCalculationApp.setAgeCalculator
                                (ageCalculator););
                AgeCalculationApp.main(null);
        }
}
```

— MAINTENANCE PROBLEM

Service Locator



Whiteboard Component Model



Dependency Injection

